

A3-PS003-TC309

Federated Traffic-Flow Prediction with Communication-Cost Evaluation

Ankitha M G
ankithamg.cs26@bmsce.ac.in

AIORI Hackathon 3

October 14, 2026

Problem Description

Centralised traffic forecasting is bandwidth-hungry and privacy-sensitive; can federated learning help?

- **Bandwidth:** edge cameras and roadside sensors must stream raw data continuously to a central cloud, consuming high cellular bandwidth.
- **Privacy:** a central cloud concentrates private travel logs in a single place.
- **Federated Learning (FL)** trains local models on edge devices and shares only weight updates with a server.
- **Open question:** how well does a compact forecaster trained with Federated Averaging (FedAvg) cope with **non-IID** traffic patterns across intersections, and what does each training round cost in bytes?
- **This project:** a federated simulator on one public traffic dataset split across 3–5 virtual intersections, comparing centralised vs. federated and IID vs. non-IID training. We report MAE, RMSE, communication bytes per round and convergence over **3 random seeds**.
- **Scope:** simulation only – no live cameras, no city-wide deployment, no formal privacy guarantees.

Background

Concepts needed to understand the problem and the proposed evaluation

- **Traffic-flow forecasting:** predict the vehicle count at a sensor/intersection from its recent history (e.g. the last 12 five-minute readings). Accuracy is measured by MAE and RMSE.
- **FedAvg [1]:** each round the server sends the global weights w to the clients; every client runs E local epochs on its own data and returns its weights w_k; the server averages them, weighted by local sample count n_k/n. Raw data never leaves a client.
- **Non-IID data:** intersections differ in traffic volume and daily peak patterns. Such heterogeneity causes client drift and slower, noisier convergence [2, 3].
- **Communication cost:** per round ≈ 2 × K clients × $|w|$ × bytes per parameter (download + upload). On cellular links, accuracy *per byte* matters as much as accuracy per round.
- **Data leakage:** traffic is a time series, so random train/test splits leak the future into training. A **chronological** split is required.

Literature & RFC Review

Only the most relevant proposals, studies and datasets are listed

Work	Contribution	Use in this project
McMahan et al. [1]	Introduces FedAvg: multiple local steps per round cut communication rounds.	Core algorithm and reference implementation.
Hsu et al. [2]	Shows performance degrades as client data distributions diverge.	Motivates the IID vs. non-IID comparison.
Li et al. (FedProx) [3]	Proximal term to stabilise training under heterogeneity.	Context only; out of scope (FedAvg required).
Liu et al. (FedGRU) [4]	Federated GRU for traffic-flow prediction.	Supports a compact GRU forecaster.
Guo et al. (ASTGCN) [5]	Provides the public PeMSD4 traffic-flow benchmark.	Candidate public dataset.

Gap: prior work focuses on accuracy; here we report accuracy *against bytes transmitted* under leakage-free chronological splits, IID and non-IID partitions, and 3 seeds. No RFC is directly applicable (standards-track protocols are not in scope).

Proposed Solution

A single-machine federated simulator with centralised and federated pipelines

- **Data:** one public traffic-flow dataset (e.g. PeMSD4 [5], 5-min counts). Window: 12 past steps → next step.
- **Split:** chronological train / val / test (60/20/20). Normalisation statistics from train only.
- **Clients:** K = 5 virtual intersections. **IID:** windows shuffled uniformly across clients. **Non-IID:** each client gets a distinct group of sensors (clustered by mean flow / daily profile).
- **Model:** 1-layer GRU (hidden 32) + linear head, ≈ 3.4k parameters ≈ 13.6 KB in float32.
- **Baselines:** centralised (pooled data) and FedAvg-IID vs. FedAvg-non-IID.
- **Metrics:** MAE, RMSE, bytes/round, rounds-to-target, over seeds 0, 1, 2.

Pseudocode: FedAvg round loop

```
w ← init(seed)
for t = 1 .. T:
  send w to K clients      # down
  for each client k:
    w_k ← LocalSGD(w, D_k, E, lr)
    upload w_k             # up
  w ← Σ_k (n_k/n)·w_k   # FedAvg
  bytes_t ← 2·K·|w|·sizeof(dtype)
  log val/test MAE, RMSE, bytes_t
report: metrics vs. round and
        metrics vs. cumulative bytes
```

Optimization Proposed

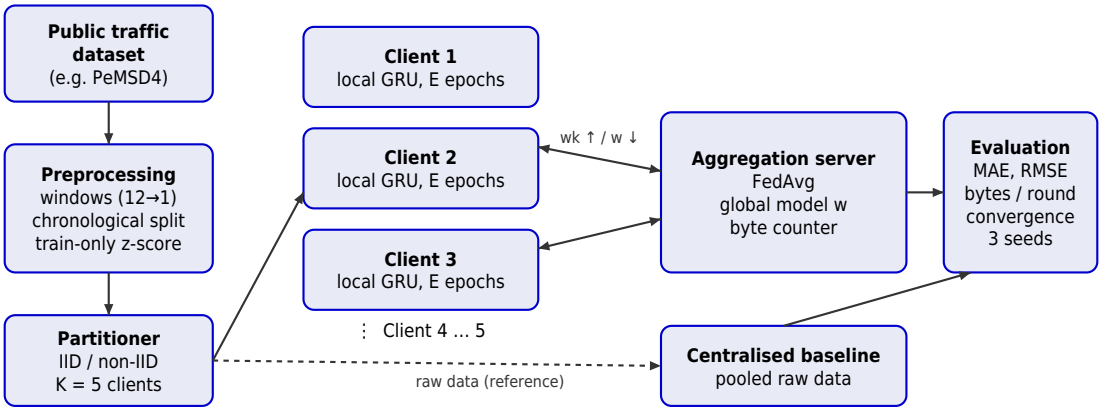
Design choices that reduce communication while protecting accuracy (before vs. after)

Aspect	Before (naive)	After (proposed)
Model size	Large LSTM/GCN, ≥100k params (≈400 KB per transfer)	Compact GRU, ≈3.4k params (≈13.6 KB per transfer)
Local work	E = 1 local epoch: many rounds to converge	E tuned (1 vs. 5): fewer rounds to reach target MAE [1]
Aggregation	Unweighted mean of client models	Sample-weighted average n_k/n
Payload precision	float32 weights	Optional float16 payload: ≈ half the bytes, accuracy checked
Stopping rule	Fixed number of rounds	Early stop on validation MAE: saves cumulative bytes
Data handling	Random split, global normalisation (leaks future)	Chronological split, train-only normalisation

Design targets and estimates (float32 = 4 bytes/parameter); measured values go in the final report.

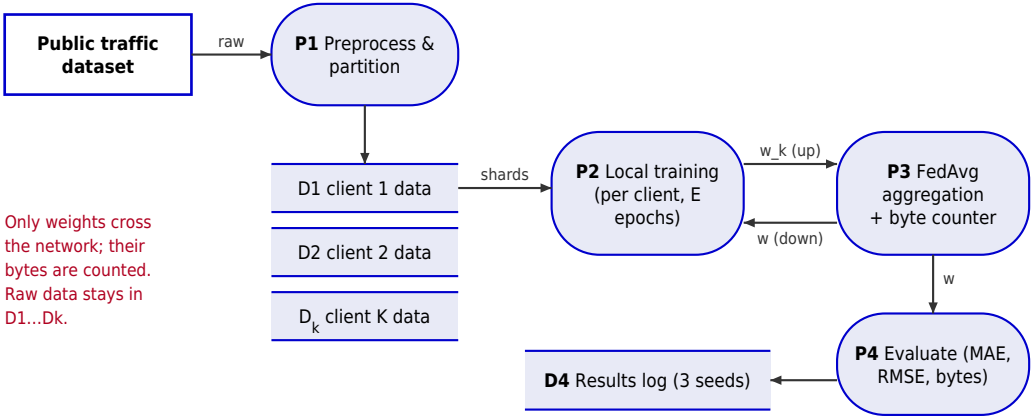
Solution Architecture & Design

Visual: pipeline from public dataset to communication-cost evaluation



Data Flow Diagram

Overview of the data and weight flows; raw data never leaves a client



Deployment Timeline

Milestones and methodology (simulation only; no field deployment)

Milestone	Work	Output
M1	Repo setup, dataset download, windowing, chronological split	Clean data pipeline
M2	Compact GRU + centralised baseline	Reference MAE/RMSE
M3	FedAvg simulator with IID partition, byte counter	Working simulator
M4	Non-IID partition; tune E, rounds; float16 option	IID vs. non-IID results
M5	Runs over 3 seeds; accuracy-vs-bytes and convergence plots	Final results tables
M6	Evaluation report, PDF upload, prototype demo with pseudocode to mentor	Submission

Methodology: incremental development in a private GitHub repository shared with the mentor (aiori-hackathon); fixed seeds 0, 1, 2 for reproducibility; every milestone is committed and reviewed.

Bibliography

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. Agüera y Arcas. “Communication-Efficient Learning of Deep Networks from Decentralized Data.” In: *Proc. AISTATS*, PMLR 54 (2017), pp. 1273–1282.

[2] T.-M. H. Hsu, H. Qi, M. Brown. “Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification.” arXiv:1909.06335 (2019).

[3] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith. “Federated Optimization in Heterogeneous Networks.” In: *Proc. MLSys* 2 (2020), pp. 429–450.

[4] Y. Liu, J. J. Q. Yu, J. Kang, D. Niyato, S. Zhang. “Privacy-Preserving Traffic Flow Prediction: A Federated Learning Approach.” In: *IEEE Internet of Things Journal* 7.8 (2020), pp. 7751–7763.

[5] S. Guo, Y. Lin, N. Feng, C. Song, H. Wan. “Attention Based Spatial-Temporal Graph Convolutional Networks for Traffic Flow Forecasting.” In: *Proc. AAAI* 33.1 (2019), pp. 922–929.

Thank You for Your Attention!